

V-Commerce CRM 360

Documentação Técnica do Projeto — RocketLab 2026

Equipe 404HexaNotFound

Metodologia de Documentação

Este documento combina três frameworks, cada um aplicado à área mais adequada do projeto:

- **Diátaxis** - estrutura as seções em quatro quadrantes (Explanation, Reference, How-to, Tutorial), evitando que conteúdo conceitual se misture com instruções operacionais.
- **Data Cards** - cada tabela Silver é documentada no formato problema → decisão → resultado.
- **Model Card** - a seção do Agente de IA documenta escopo, exemplos validados, guardrails e limitações.

Quadrante	Seções
Diátaxis: Explanation	1. Visão Geral
Diátaxis: Reference	2. Eng. de Dados · 3. Backend · 4. Frontend
Diátaxis: Tutorial	5. Agente de IA
Diátaxis: How-to	6. Como Executar
Data Card	2. Eng. de Dados (por tabela)
Model Card	5. Agente de IA

1. Visão Geral do Projeto

Diátaxis: Explanation

O V-Commerce CRM 360 é uma plataforma integrada desenvolvida para a V-Commerce, varejista digital brasileira com mais de 50.000 clientes e 300.000 pedidos acumulados. O projeto centraliza dados dispersos de múltiplas áreas em uma única fonte confiável de verdade, exposta por meio de dashboards analíticos e de um agente de IA conversacional capaz de responder perguntas de negócio em linguagem natural.

Problema que motivou o projeto

Cada área da V-Commerce trabalhava com sua própria visão de dados: a equipe comercial em planilhas, o suporte sem contexto do cliente, o time de produto sem visibilidade de desempenho individual. Sem uma fonte única de verdade, qualquer análise exigia intervenção técnica e levava dias.

Módulos entregues

Módulo	Tecnologias	Responsabilidade
Engenharia de Dados	Databricks · PySpark · Delta Lake	Pipeline Bronze → Silver → Gold
Backend (API)	Python · FastAPI · SQLite · Pydantic · SQLAlchemy	Endpoints REST para CRM e Agente
Frontend (CRM)	React · TypeScript · Vite · Tailwind CSS · Chart.js	Interface web com dashboards e chat
Agente de IA	Gemini 3.1 Flash-Lite · Pydantic AI	Consultas em linguagem natural e Text-to-SQL

2. Engenharia de Dados

Diátaxis: Reference · Data Card

O pipeline segue a Arquitetura Medalhão em três camadas executadas no Databricks com PySpark e armazenamento em formato Delta Lake. Cada camada tem responsabilidade exclusiva e nenhuma reescreve dados da anterior.

2.1 Camada Bronze — Ingestão com rastreabilidade

Ingere os seis arquivos CSV brutos como tabelas Delta sem nenhuma transformação de conteúdo. Um timestamp de ingestão é adicionado a cada linha, garantindo rastreabilidade e base para auditoria e reprocessamento.

Tabela Delta	Arquivo de origem
bronze.tb_clientes	clientes.csv
bronze.tb_pedidos	pedidos.csv

Tabela Delta	Arquivo de origem
bronze.tb_catalogo_produtos	catalogo_produtos.csv
bronze.tb_suporte_tickets	suporte_tickets.csv
bronze.tb_avaliacoes	avaliacoes.csv
bronze.tb_clickstream	clickstream.csv

2.2 Camada Silver — Limpeza e padronização (Data Cards por tabela)

Cada tabela passou por diagnóstico individual antes de qualquer transformação. Os problemas foram classificados por criticidade e as decisões registradas com justificativa de negócio.

Clientes

- **Problema:** linhas duplicadas por id_cliente. **Decisão:** deduplicação via Window Function particionada por id_cliente, mantendo o registro mais recente pela data_cadastro.
- **Problema:** e-mails sem domínio ou sem @. **Decisão:** validação por regex; quando um email era inválido, mas o cliente possuía nome, telefone e cidade/estado, os emails eram marcados como nulos em vez de removidos.
- **Problema:** telefone em 6 formatos diferentes com sufixos "r. XXXX". **Decisão:** separação em telefone_formatado (padrão (xx)xxxx-xxxx) e telefone_ramal; remoção do prefixo +55.
- **Problema:** endereço com complemento embutido (ex: "3577 Anita Knoll Apt. 107"). **Decisão:** decomposição em logradouro, número e complemento via regex para padrões BR e USA.
- **Problema:** datas de nascimento fora da faixa 1920–2020 ou posteriores ao cadastro. **Decisão:** nulificação das datas inválidas; dados do cliente preservados.
- **Problema:** device IDs concatenados por ponto-e-vírgula. **Decisão:** normalização em tabela 1:N (id_cliente × id_device).

Pedidos

- **Problema:** possibilidade de id_pedido duplicado com colunas iguais ou divergentes. **Decisão:** implementação de validação para identificação de duplicatas exatas e divergentes; como nenhuma inconsistência foi encontrada nos dados analisados, não foi necessário aplicar tratamento corretivo.
- **Problema:** possibilidade de chaves estrangeiras sem referência válida. **Decisão:** implementação de validação de id_cliente e id_produto em relação às tabelas de origem; como não foram encontrados registros inválidos, nenhum tratamento adicional precisou ser realizado.

- **Problema:** valor_pedido como string com "R\$" e vírgula decimal. **Decisão:** conversão para float com remoção de "R\$" e troca de vírgula por ponto.
- **Problema:** datas futuras na coluna data_pedido. **Decisão:** datas após 28/04/2026 convertidas para nulo.
- **Problema:** método_pagamento e status com variações de capitalização e abreviações. **Decisão:** padronização via dicionário de substituições em minúsculas.

Catálogo de Produtos

- **Problema:** campo ativo com 12 variações (Sim, sim, SIM, S, 1, Nao, NAO, N, 0...). **Decisão:** padronização para booleano True/False.
- **Problema:** preços negativos (-100), zeros e 23 nulos falsos (string "null"). **Decisão:** negativos convertidos para valor absoluto; zeros substituídos pelo preço de referência calculado a partir dos pedidos ($\text{valor_pedido} \div \text{quantidade}$); campo convertido para Double.
- **Problema:** categoria com typos, abreviações, acentuação inconsistente e 27 nulos falsos. **Decisão:** mapeamento para 8 categorias canônicas (Eletronicos, Moveis, Brinquedos, Beleza, Esportes, Automotivo, Casa, Vestuario); categoria inferida para os casos sem valor.
- **Problema:** 14 produtos ativos com estoque zero. **Decisão:** marcados automaticamente como inativos.

Suporte

- **Problema:** tipo_problema com erros de digitação ("p4gamento"), abreviações ("pag") e mistura de idiomas ("PAYMENT"). **Decisão:** normalização para categorias canônicas em português.
- **Problema:** data_resolucao com 2.072 nulos legítimos (tickets em aberto). **Decisão:** criação da coluna status_ticket (Aberto/Fechado) a partir dos nulos; registros preservados.
- **Problema:** tempo_resolucao_horas inconsistente com os timestamps de abertura e resolução. **Decisão:** descarte dos valores de origem e recálculo a partir da diferença exata em segundos entre os timestamps, convertida para horas.

Avaliações

- **Problema:** nota_produto contendo strings e valores fora do intervalo 1–5. **Decisão:** padronização da coluna para valores inteiros no intervalo válido; valores maiores que 5 foram mapeados para 5 e valores menores que 1 foram mapeados para 1. Valores textuais foram convertidos semanticamente (ótimo → 5, bom → 4, ruim → 2, péssimo → 1), seguida da conversão da coluna para tipo Inteiro.
- **Problema:** nota_nps com valores acima de 10 (11) e abaixo de 1 (0). **Decisão:** restrição ao intervalo 1–10; coluna convertida para Inteiro.
- **Problema:** recomenda com 8 variações (s, yes, SIM, n, no, NAO, 1, 0). **Decisão:** padronização para sim/nao.

- **Problema:** datas futuras e nulas na coluna data_avaliacao. **Decisão:** linhas removidas; formato unificado para yyyy-MM-dd.

Clickstream

- **Problema:** 6.367 checkouts/purchases sem id_cliente e 15.259 sem id_produto. **Decisão:** linhas críticas removidas (compra sem saber quem comprou ou o que foi comprado é dado inválido).
- **Problema:** nulos em id_cliente e id_produto em eventos normais (login, search, page_view). **Decisão:** criação de flags booleanas flag_anonimo e flag_sem_produto para rastreio de nulos esperados sem alterar os dados originais.
- **Problema:** combinações inválidas entre canal e dispositivo. **Decisão:** app + desktop → dispositivo corrigido para mobile; web + mobile → canal corrigido para mobile_web.
- **Problema:** tempo_pagina_seg com ~9% de nulos, zeros e negativos. **Decisão:** substituição pela mediana dos valores válidos (preferível à média por não ser sensível a outliers).

2.3 Camada Gold — Tabelas analíticas consolidadas

A Gold materializa a fonte única de verdade. Cada tabela é orientada a um caso de uso específico e otimizada para leitura.

Tabela Gold	Granularidade	Alimenta
gold.v_cliente_360	1 linha por cliente	Perfil 360, Busca, Agente de IA
gold.desempenho_produtos	1 linha por produto	Catálogo com métricas, Agente
gold.kpi_por_categoria	Ano/mês por corte	Dashboard de KPIs, Agente
gold.kpi_por_estado	Ano/mês por corte	Dashboard de KPIs, Agente
gold.kpi_por_status	Ano/mês por corte	Dashboard de KPIs, Agente
gold.analise_tickets	1 linha por ticket	Visualização de tickets, Catálogo
gold.comportamento_digital	1 linha por cliente	Perfil 360 enriquecido, Agente
gold.pedidos_por_cliente	1 linha por pedido	Visualização de pedidos
gold.historico_avaliacoes	1 linha por avaliação	Dashboard

Decisão de design: a tabela `v_cliente_360` consolida métricas de cinco origens diferentes (pedidos, avaliações, tickets, clickstream e cadastro) em uma única linha por cliente via `left joins` sucessivos. Campos de agregação numérica utilizam valor 0 quando não há histórico associado, garantindo consistência em consultas analíticas e evitando valores nulos em métricas acumuladas. Já campos descritivos, como `metodo_pagamento_preferido`, podem permanecer nulos quando não existe informação disponível. A coluna `produto_mais_visitado` substitui valores nulos por "Produto Indisponível" para padronizar a visualização no CRM.

2.4 Workflow agendado

O pipeline completo (Bronze → Silver → Gold) é orquestrado por um workflow no Databricks com execução automática diária às 18:00. O arquivo `workflow.yaml` define a sequência e as dependências entre os três notebooks, eliminando intervenção manual.

3. Backend — API REST (FastAPI)

Diátaxis: Reference

O backend expõe os dados da camada Gold por meio de uma API REST construída com FastAPI e Python. O banco de dados local é um SQLite populado a partir das tabelas Gold, permitindo execução do CRM sem dependência de nuvem em ambiente de demonstração.

3.1 Organização do código

Camada	Pasta	Responsabilidade
Routers	<code>app/routers/</code>	Definição dos endpoints e validação de entrada/saída
Schemas	<code>app/schemas/</code>	Contratos Pydantic para request e response
Models	<code>app/models/</code>	Mapeamento das tabelas SQLite via SQLAlchemy
Database	<code>app/database.py</code>	Configuração da conexão e sessão com o banco
Testes	<code>tests/</code>	Testes automatizados por domínio (pytest)

3.2 Endpoints disponíveis

Prefixo	Domínio	Operações
/clientes	Clientes	Listagem, filtros, detalhe individual (perfil 360)
/produto	Produtos	Listagem, filtros, CRUD completo
/pedidos_cliente	Pedidos	Listagem com filtros por status, data e cliente
/ticket	Suporte	Listagem, filtros por tipo, status e agente
/kpi-state	KPIs por estado	Métricas agregadas por UF e período
/kpi-category	KPIs por categoria	Métricas agregadas por categoria de produto
/kpi-status	KPIs por status	Métricas agregadas por status de pedido
/agent/chat	Agente de IA	Recebe pergunta, retorna SQL + resultado
/agent/clear	Agente de IA	Limpa o histórico da conversa
/avaliacoes	Histórico de avaliações	Lista de avaliações e métricas do gráfico de avaliações
/health	Monitoramento	Verificação de disponibilidade da API

3.3 Testes automatizados

Os principais domínios da aplicação possuem arquivos de testes dedicados, como `test_cliente.py`, `test_pedidos.py`, `test_produto.py`, `test_ticket.py` e `test_kpi.py`. Os testes implementados verificam tanto cenários de sucesso quanto casos de erro esperados, utilizando um banco de dados de teste configurado via `conftest.py`.

4. Frontend — CRM Web (React + TypeScript)

Diátaxis: Reference

O frontend é uma Single Page Application construída com React, TypeScript e Vite, seguindo a metodologia Atomic Design na organização dos componentes.

4.1 Arquitetura de componentes

Nível	Pasta	Exemplos
Atoms	components/atoms/	KpiCard, StatusBadge, SearchInput, ChatBubble
Molecules	components/molecules/	FilterBar, TableRow, BotMessage, SuggestionList
Organisms	components/organisms/	Navbar, DataTable, ProductGrid, ChatbotOverlay
Pages	components/pages/	Dashboard, ClientsPage, OrdersPage, SupportPage
Hooks	hooks/	useDashboard, useClienteDetalhe, useOrders
Types	components/types/	cliente.types.ts, produto.types.ts, pedido.types.ts

4.2 Telas entregues

Dashboard de KPIs: visão executiva com receita, total de pedidos e ticket médio. Filtros por período, categoria, status de pedido e estado permitem comparar métricas entre diferentes recortes, com gráficos de evolução temporal para análise de tendências.

Busca e filtro de clientes: listagem paginada com filtros por nome, e-mail, cidade, estado e segmento. Ponto de entrada para o Perfil 360.

Perfil 360 do cliente: tela de detalhe com histórico completo — pedidos, tickets abertos, avaliações e segmento calculado. A visão também apresenta métricas comportamentais e de

engajamento, como taxa de compra de produtos, taxa de abandono de carrinho e canal predominante de interação do cliente.

Listagem de pedidos: tabela com filtros por status, método de pagamento e período, permitindo segmentar pedidos em diferentes recortes temporais. A funcionalidade auxilia na investigação de anomalias operacionais diretamente no CRM.

Catálogo com métricas: visão por produto com nota média, receita, total de tickets e flag de alto índice de suporte. A interface também permite ordenação por diferentes critérios, como receita, estoque disponível, quantidade vendida, preço e avaliação dos produtos.

Página de detalhe do produto: visão consolidada com métricas de receita, crescimento mensal, pedidos, NPS, taxa de tickets e visualizações. A tela também apresenta evolução temporal, distribuição das avaliações por estrelas e tickets vinculados ao produto, facilitando análises operacionais e de satisfação do cliente.

Tickets de suporte: listagem de tickets, incluindo filtros por status do ticket, categoria do problema e agente de suporte.

CRUD de produtos: formulário para adição, edição e remoção de produtos.

Chat de agente de IA: interface acessível por um balão flutuante integrado ao CRM, permitindo iniciar conversas e consultas analíticas sem sair da tela atual. O recurso oferece acesso rápido a informações, métricas e perguntas em linguagem natural diretamente durante a navegação do sistema.

4.3 Funcionalidades diferenciais

Exportação CSV: qualquer listagem pode ser exportada em CSV diretamente pelo navegador, eliminando dependência de TI para análises externas.

Processamento de clickstream integrado à visão 360 do cliente: eventos de navegação, visualizações de produtos e interações no site são consolidados ao perfil do cliente, permitindo análises comportamentais mais completas e enriquecendo métricas de engajamento, interesse e conversão dentro do CRM.

5. Agente de Inteligência Artificial

Diátaxis: Tutorial · Model Card

O agente é um sistema Text-to-SQL integrado ao CRM que converte perguntas em linguagem natural em queries SQL válidas, executa-as sobre o banco local e retorna os resultados

diretamente na interface. O modelo base é o “gemini-3.1-flash-lite-preview” (Google), acessado via API.

5.1 Arquitetura do agente

Arquivo	Responsabilidade
agent.py	Função principal perguntar(): orquestra a construção do prompt, chamada ao GenAI, validação e execução do SQL, além do registro do histórico da sessão.
prompt.py	System prompt com schema completo das tabelas do banco, exemplos few-shot de perguntas de negócio e regras obrigatórias para geração e execução segura de queries SQL.
database.py	Conexão com SQLite via SQLAlchemy e funções get_schema() e execute_query(sql) para acesso e execução de queries no banco.
session_memory.py	Gerenciamento do histórico de conversa por session_id para contexto de perguntas encadeadas, com funções de construção, registro e limpeza da sessão.
context_enricher.py	Seleção semântica de tabelas e colunas relevantes, com coleta de amostras para enriquecimento de contexto e fallback heurístico local.
utils.py	Validação do SQL gerado, extração e verificação das tabelas referenciadas, além de helpers de execução e parsing de queries.
/tests/*	Suíte de testes automatizados para validação de queries, regras de segurança e funções auxiliares do agente.

5.2 Fluxo de uma pergunta

1. **Entrada:** o usuário digita uma pergunta em português no chat do CRM.

2. **Contexto de sessão:** o histórico da conversa é recuperado e adicionado ao prompt para suporte a perguntas encadeadas.
3. **Contexto enriquecido:** o agente adiciona amostras e informações auxiliares sobre tabelas relevantes para a consulta.
4. **Geração de SQL:** o modelo GenAI recebe o system prompt (schema + exemplos + regras) e retorna um objeto JSON com o SQL candidato e metadados de validação.
5. **Validação:** apenas queries SELECT/WITH são permitidas; tabelas inválidas, múltiplas queries e comandos perigosos são bloqueados.
6. **Execução:** se válido, o SQL é sanitizado e executado no banco SQLite, retornando os resultados ao usuário.
7. **Guardrail/Recusa:** perguntas fora do escopo ou solicitações operacionais são recusadas sem geração de respostas inventadas.

5.3 Diferenciais do agente

Memória de conversa: perguntas encadeadas funcionam sem repetir contexto. Um follow-up como "e quantos desses fizeram mais de 3 compras?" após uma pergunta anterior é interpretado corretamente.

Sugestões de perguntas: ao abrir o chat, o usuário recebe sugestões pré-definidas que exemplificam o que é possível perguntar, reduzindo a barreira de entrada para usuários não técnicos.

Integração nativa: o chat é acessado pelo ChatbotOverlay diretamente no CRM, sem troca de contexto. O endpoint /agent/chat é exposto pelo backend FastAPI.

Guardrails no agente: perguntas fora do escopo do CRM, solicitações potencialmente problemáticas ou consultas incompatíveis com os dados disponíveis são identificadas e tratadas com respostas controladas, evitando comportamentos inesperados do agente.

6. Como Executar o Projeto

Diátaxis: How-to

O projeto possui três módulos independentes. O backend deve ser iniciado antes do frontend.

Backend (FastAPI)

1. Entrar na pasta backend/
2. Criar e ativar ambiente virtual Python
3. Instalar dependências: `pip install -r requirements.txt`
4. Iniciar servidor: `python -m app.main`

5. API disponível em <http://localhost:8000>

Frontend (React)

1. Entrar na pasta frontend/
2. Instalar dependências: `pnpm install`
3. Iniciar em modo desenvolvimento: `pnpm run dev`
4. Interface disponível em <http://localhost:5173>

Agente de IA (standalone)

1. Entrar na pasta ai-agent/
2. Criar e ativar ambiente virtual Python
3. Copiar `.env.example` para `.env` e inserir a `GOOGLE_API_KEY`
4. Chave obtida em: aistudio.google.com/app/apikey
5. Executar testes: `python -m pytest`